

Documentación entrega 3 FBD

Santiago Tarazona - 202513771

Andres Betancur - 202510228

Mateo Pardo - 202510502

1. Introducción:

Este proyecto tiene como objetivo principal implementar un sistema de reseñas para la cadena hotelera Dann-Alpes utilizando MongoDB como base de datos documental y Oracle/APEX como sistema relacional principal. La funcionalidad desarrollada permite almacenar y consultar reseñas realizadas por los clientes sobre sus estadías en los hoteles, adicionalmente, permite manejar votos de utilidad, respuestas de administradores y reseñas destacadas.

Se decidió utilizar MongoDB debido a la flexibilidad que ofrece el modelo documental para representar información de diversas estructuras, mientras que Oracle continúa administrando la información transaccional principal del sistema, como hoteles, reservas y clientes.

La arquitectura general del sistema quedó conformada de la siguiente manera:

- Oracle/APEX: manejo de clientes, hoteles, reservas y lógica relacional.
- MongoDB: almacenamiento documental de las reseñas.
- FastAPI: capa intermedia entre APEX y MongoDB.

Nota: Es necesario hacer el proceso de agregar los componentes de autenticación del proyecto 2.

2. Workload:

2.1 Cantidad estimada de entidades:

Entidad	Cantidad estimada
Hoteles	15
Clientes	50.000
Reservas	250.000
Reseñas	250.000
Respuestas de admin	150.000
Votos útiles	500.000

Las cantidades estimadas se basan en el comportamiento esperado del sistema y en el volumen de usuarios planteado en el enunciado del proyecto.

2.2 Análisis de operaciones de lectura y escritura

Entidad	Operación	Información accedida conjuntamente	Tipo
Reseñas	Consultar reseñas de un hotel	Reseña + utilidad + respuesta administrador	Lectura
Reseñas	Consultar historial de reseñas propias	Reseña + estado + utilidad	Lectura
Reseñas	Crear reseña	Calificación + comentario + reserva	Escritura
Reseñas	Editar reseña	Calificación + comentario	Escritura
Reseñas	Eliminar reseña	Estado reseña	Escritura
Reseñas	Marcar reseña como útil	Utilidad + usuarios	Escritura
Reseñas	Responder reseña	Respuesta administrador	Escritura

Reseñas	Consultar reseñas destacadas	Reseña + destacada + utilidad	Escritura
Reseñas	RFC1 – Top hoteles	Calificación + hotel	Lectura
Reseñas	RFC2 – Evolución reputación	Calificación + fecha	Lectura
Reseñas	RFC3 – Comparación hoteles	Calificación + respuestaAdmin + destacada	Lectura

A partir del análisis realizado se concluyó que el sistema presenta una alta cantidad de lecturas sobre las reseñas y una cantidad moderada de escrituras.

Debido a esto, se decidió utilizar estructuras embebidas para almacenar información que normalmente se consulta junto con la reseña, reduciendo así la cantidad de consultas adicionales.

2.3 Cuantificación de operaciones de lectura y escritura

Entidad	Operación	Tipo	Frecuencia estimada
Reseñas	Consultar reseñas de un hotel	Lectura	10.000/día
Reseñas	Crear reseña	Escritura	300-500/día
Reseñas	Editar respuesta administrador	Escritura	200-300/día
Reseñas	Registrar voto útil	Escritura	500/día
Reseñas	RFC analíticos	Lectura	30/día
Clientes	Consultar cliente	Lectura	5/min
Reservas	Validar reserva completada	Lectura	300-500/día

A partir de este análisis se concluyó que la mayor carga del sistema corresponde a consultas frecuentes sobre reseñas, por lo que se priorizó optimizar las lecturas mediante estructuras embebidas.

3. Modelo conceptual NoSQL

A partir del análisis del workload y de las reglas de negocio del sistema Dann-Alpes, se definió una colección principal llamada "resenas" que almacena la información relacionada con las opiniones de los huéspedes. Las entidades Hotel, Cliente y Reserva continúan existiendo en Oracle y son referenciadas desde MongoDB mediante identificadores compartidos.

3.1 Entidades del sistema

Entidad	Descripción
Reseña	Comentario y calificación realizada por un cliente sobre una reserva
Respuesta admin	Respuesta oficial realizada por un administrador sobre una reseña
Utilidad	Información relacionada con votos útiles realizados por usuarios
Hotel	Hotel sobre el cual se realiza la reseña
Cliente	Cliente que escribe la reseña
Reserva	Reserva asociada a la reseña

3.2 Relaciones y cardinalidades

Relación	Cardinalidad	Tipo de asociación
Hotel - Reseña	1 a muchos	Referenciada
Cliente - Reseña	1 a muchos	Referenciada
Reserva - Reseña	1 a 0/1	Referenciada

Reseña - respuestaAdmin	1 a 0/1	Embebida
Reseña - Utilidad	1 a muchos	Embebida

3.3 Selección del esquema de asociación

Relaciones referenciadas

Las entidades Hotel, Cliente y Reserva se mantuvieron referenciadas debido a que su información principal ya se encuentra almacenada en Oracle.

Por esta razón, en MongoDB únicamente se almacenan los identificadores:

- idHotel
- idCliente
- idReserva

Esto evita duplicación innecesaria de información y mantiene consistencia con el modelo relacional.

Relaciones embebidas

La información correspondiente a:

- respuestaAdmin
- utilidad

fue almacenada de manera embebida debido a que siempre se consulta junto con la reseña principal.

Este diseño permite disminuir la cantidad de accesos a la base de datos y simplifica las consultas frecuentes.

4. Modelo documental en JSON

A continuación se presenta un ejemplo del documento utilizado para representar una reseña dentro de MongoDB:

```
idReserva : 12
idHotel : 12
idCliente : 12
calificacion : 3
comentario : "Reseña número 12. La experiencia general en el hotel fue satisfactoria..."
fechaCreacion : 2025-01-10T00:00:00.000+00:00
fechaActualizacion : 2025-01-11T00:00:00.000+00:00
estado : "publicada"
destacada : false
▾ utilidad : Object
  cantidad : 6
  ▾ usuarios : Array (6)
    0: 13
    1: 14
    2: 15
    3: 16
    4: 17
    5: 18
  ▾ respuestaAdmin : Object
    idAdmin : 12
    texto : "Gracias por compartir su experiencia con nuestro hotel."
    fechaRespuesta : 2025-01-15T00:00:00.000+00:00
```

En el documento anterior se evidencia el uso de referencias mediante identificadores y de Información embebida para utilidad y respuesta administrativa.

5. Creación de colecciones y validaciones

Se creó la colección principal llamada “resenas” y se realizó utilizando Mongo Shell.

Adicionalmente, se implementó un esquema de validación utilizando JSON Schema para garantizar integridad básica sobre los documentos almacenados.

Las principales validaciones implementadas fueron:

- idReserva obligatorio.
- idHotel obligatorio.
- idCliente obligatorio.
- calificación entre 1 y 5.
- comentario obligatorio.

- estado restringido a “publicada” o “eliminada”.
- validación de fechas.

6. Población de datos

La base de datos fue poblada mediante scripts en MongoDB.

Se insertaron aproximadamente:

- 60 reseñas.
- Diferentes hoteles.
- Distintos meses del año.
- Calificaciones variadas.
- Reseñas destacadas.
- Respuestas administrativas.
- Votos útiles.

Esto permitió probar adecuadamente las consultas analíticas requeridas.

7. Consultas RFC

RFC1 – Top hoteles por calificación promedio

Esta consulta permite obtener los hoteles con mejor calificación promedio dentro de un rango de fechas determinado.

La consulta realiza:

- filtrado por fechas.
- agrupamiento por hotel.
- cálculo de promedio.
- ordenamiento descendente.

Pipeline implementado:

```
use("dann_alpes_nosql");

db.resenas.aggregate([

  {

    $match: {

      fechaCreacion: {

        $gte: new Date("2025-01-01"),

        $lte: new Date("2025-12-31")

      },

      estado: "publicada"

    }

  },

  {

    $group: {

      _id: "$idHotel",

      promedioCalificacion: {

        $avg: "$calificacion"

      },

      totalResenas: {

        $sum: 1

      }

    }

  }

])
```



```
    }

    }

  },

  {

    $sort: {

      promedioCalificacion: -1

    }

  },

  {

    $limit: 10

  }

];
```

RFC2 – Evolución de la reputación de un hotel

Esta consulta permite observar la evolución mensual de la reputación de un hotel durante un año específico.

La consulta recibe como parámetros:

- hotel seleccionado.
- año seleccionado.

pipeline implementado:

```
db.resenas.aggregate([

  {

    $match: {

      idHotel: idHotelParam,

      estado: "publicada",

      fechaCreacion: {

        $gte: new Date(`${anioParam}-01-01`),

        $lt: new Date(`${anioParam + 1}-01-01`)

      }

    }

  },

  {

    $group: {

      _id: {

        mes: {

          $month: "$fechaCreacion"

        }

      }

    }

  },
```

```
    promedioMensual: {  
      $avg: "$calificacion"  
    },  
  
    cantidadResenas: {  
      $sum: 1  
    }  
  }  
},  
  
{  
  $project: {  
  
    _id: 0,  
  
    mes: "$_id.mes",  
  
    promedioMensual: {  
      $round: ["$promedioMensual", 2]  
    },  
  
    cantidadResenas: 1  
  }  
},
```

```
{  
  
  $sort: {  
  
    mes: 1  
  
  }  
  
}  
  
]);
```

RFC3 – Comparación de hoteles por ciudad

Esta consulta permite comparar hoteles utilizando:

- promedio general;
- cantidad de reseñas;
- porcentaje de respuestas administrativas;
- porcentaje de reseñas destacadas.

La consulta devuelve inicialmente el identificador del hotel desde MongoDB y posteriormente la API consulta Oracle para obtener información detallada del hotel y de la ciudad.

Pipeline implementado:

```
db.resenas.aggregate([  
  
  {  
  
    $match: {  
  
      estado: "publicada"
```

```
}

},

{

  $group: {

    _id: "$idHotel",

    promedioGeneral: {

      $avg: "$calificacion"

    },

    totalResenas: {

      $sum: 1

    },

    respuestasAdmin: {

      $sum: {

        $cond: [

          {

            $ne: ["$respuestaAdmin", null]

          },

          1,

          0

        ]

      }

    }

  }

}
```

```

    }

    },

    destacadas: {

        $sum: {

            $cond: [

                {

                    $eq: ["$destacada", true]

                },

                1,

                0

            ]

        }

    }

}

] );

```

8. Arquitectura del sistema

La arquitectura final implementada fue la siguiente:

- Oracle mantiene la información relacional principal.
- MongoDB almacena las reseñas y sus estructuras documentales.
- FastAPI actúa como intermediario entre APEX y las bases de datos.
- APEX consume los endpoints expuestos por la API.

9. Escenarios de prueba

RFC1:

De uso desde 1/01/25 hasta 31/12/25 como fechas de prueba.

The screenshot shows the 'Top Hoteles' application interface. At the top, there's a navigation bar with the user 'Dann Alpes' and a search bar. Below the title 'Top Hoteles', there's a section 'Top 10 Hoteles por Calificación'. It includes date filters: 'Fecha inicio: 01/01/2025' and 'Fecha fin: 12/31/2025'. A 'Buscar' button is present. The main table has three columns: 'Hotel ID', 'Promedio', and 'Total Reseñas'. The data is as follows:

Hotel ID	Promedio	Total Reseñas
9	5	4
14	5	4
4	5	4
3	4	4
13	4	4
8	4	4
12	3	4
2	3	4
7	3	4
11	2	4

The bottom of the interface shows a status bar with 'Release 1.0' and various application controls.

RFC2:

Se introdujo el año 2025 y el hotel con id 1.

The screenshot shows the 'Evolucion Hotel' application interface. At the top, there's a navigation bar with the user 'Dann Alpes' and a search bar. Below the title 'Evolucion Hotel', there's a section 'Evolución Reputación Hotel'. It includes filters: 'Año: 2025' and 'ID Hotel: 1'. A 'Buscar' button is present. The main table has three columns: 'Mes', 'Promedio', and 'Cantidad Reseñas'. The data is as follows:

Mes	Promedio	Cantidad Reseñas
8	2	1
2	2	1
5	2	1
11	2	1

The bottom of the interface shows a status bar with 'Release 1.0' and various application controls.

RFC3:

Se probó con la ciudad de Bogotá.

The screenshot shows the 'Perfil Comparativo Hoteles por Ciudad' page. At the top, there's a search bar with 'Ciudad: Bogotá' and a 'Buscar' button. Below this is a table with the following data:

Hotel	Ciudad	Promedio	Total Reseñas	% Resp Admin	% Destacadas
Hotel Dann Alpes Bogotá	Bogotá	2.00	4	50.0%	0.0%

The bottom of the screen shows the application's footer with 'Release 1.0' and various development tools like 'App 15349', 'Page 5', 'Session', 'Debug', 'Quick Edit', 'Customize', and 'Settings'.

Requerimientos funcionales:

Reseñas:

The screenshot shows the 'Crear Reseña' page. On the left is a sidebar menu with options: Home, Crear cliente, Reserva, Reserva cliente, Reseñas Hotel, Top Hoteles, Evolucion Hotel, Perfil Comparativo Hote..., **Crear Reseña**, Editar Reseña, Eliminar Reseña, Ver Reseña Hotel, Marcar Util, and Historial Cliente. The main content area has the title 'Crear Reseña' and a form with the following fields:

- ID Reserva: 301
- ID Hotel: 5
- ID Cliente: 1
- Calificación (1-5): 5
- Comentario: gracias por todo

At the bottom of the form is a 'Crear Reseña' button and a green message: 'Resena creada correctamente'.

lacademy2.oracle.com

S4_G8_20...https://blog...Mateox18/...exámenes p...Page Desig...Editar Rese...Documenta...Cheat She...determinan...FastAPI - S...

Dann Alpes

juan.gomez1@gmail.com

Editar Reseña

Editar Reseña

ID Reserva: 1

Nueva Calificación (1-5): 2

Nuevo Comentario:
muy malo

Editar Reseña

✓ Resena actualizada correctamente

Release 1.0

App 15349Page 19SessionDebugQuick EditCustomize

lacademy2.oracle.com

S4_G8_20...https://blog...Mateox18/...exámenes p...Page Desig...Eliminar Re...Documenta...Cheat She...determinan...FastAPI - S...

Dann Alpes

juan.gomez1@gmail.com

Eliminar Reseña

Eliminar Reseña

ID Reserva: 500

Eliminar Reseña

✓ Resena eliminada correctamente

Release 1.0

App 15349Page 20SessionDebugQuick EditCustomize

lacademy2.oracle.com

S4_G8_20...https://blog...Mateox18/...exámenes p...Page Desig...Ver Reseña...Documenta...Cheat She...determinan...FastAPI - S...

Dann Alpes

juan.gomez1@gmail.com

Ver Reseña Hotel

Reseñas del Hotel

ID Hotel: 1

Buscar

ID Reserva	Calificación	Comentario	Fecha	Útiles
1	2	muy malo	2025-02-10T00:00:00	2
16	2	Reseña número 16. La experiencia general en el hotel fue satisfactoria y el servicio recibido cumplió con las expectativas del huésped.	2025-05-10T00:00:00	3
31	2	Reseña número 31. La experiencia general en el hotel fue satisfactoria y el servicio recibido cumplió con las expectativas del huésped.	2025-08-10T00:00:00	4
46	2	Reseña número 46. La experiencia general en el hotel fue satisfactoria y el servicio recibido cumplió con las expectativas del huésped.	2025-11-10T00:00:00	5
100	5	muy buen servicio	2026-05-26T22:18:47.355000	0
501	5	very gud	2026-05-26T22:52:38.743000	0

Release 1.0

App 15349Page 21SessionDebugQuick EditCustomize

lacademy2.oracle.com

S4_G8_20...https://blog...Mateox18/...exámenes p...Page Desig...Historial CS...Documenta...Cheat She...determinan...FastAPI - S...

Dann Alpes

juan.gomez1@gmail.com

Historial Cliente

Historial de Reseñas

ID Cliente: 1

Buscar

ID Reserva	Hotel	Calificación	Estado	Útiles	Respuesta Admin
1	1	2	publicada	3	No
501	1	5	publicada	0	No
502	2	4	publicada	0	No

Release 1.0

App 15349Page 24SessionDebugQuick EditCustomize

lacademy2.oracle.com

Dann Alpes admin.bogota

Responder Reseña

Responder Reseña

ID Reserva: 2

ID Admin: 1

Respuesta:

Gracias por los comentarios.

Responder

✓ Respuesta del admin guardada correctamente

Release 1.0

App 15349 Page 26 Session Debug Quick Edit Customize

lacademy2.oracle.com

Dann Alpes admin.bogota

Eliminar Resena Admin

Eliminar Reseña Admin

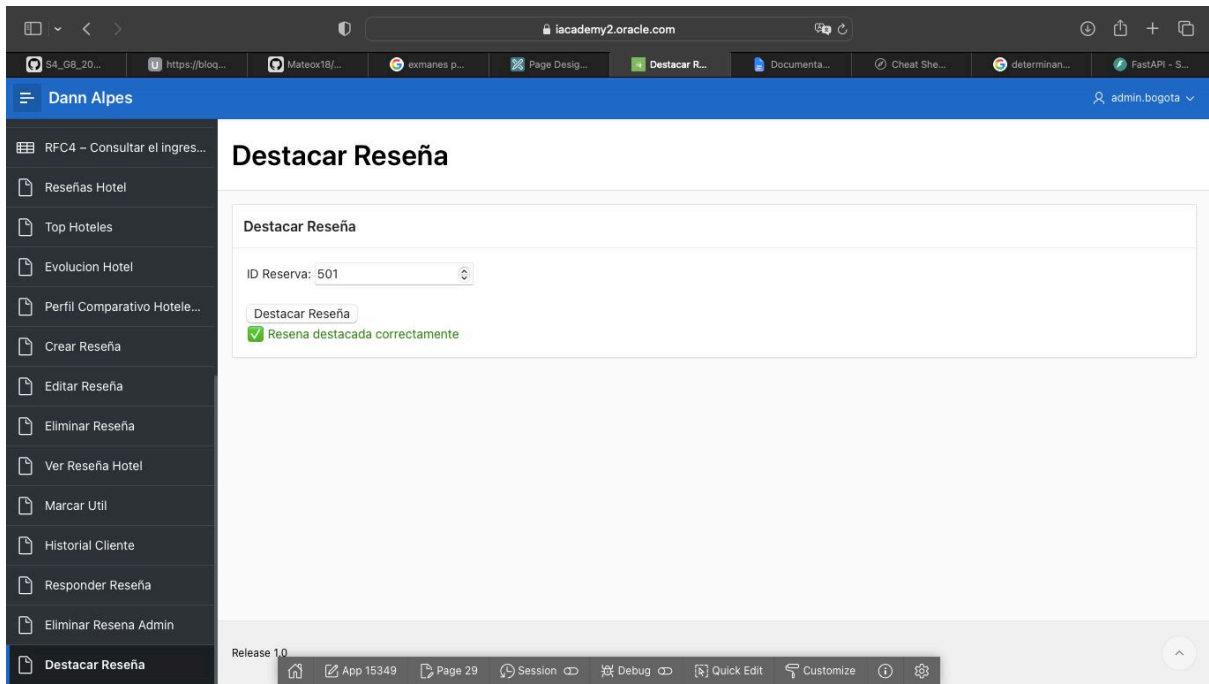
ID Reserva: 502

Eliminar Reseña

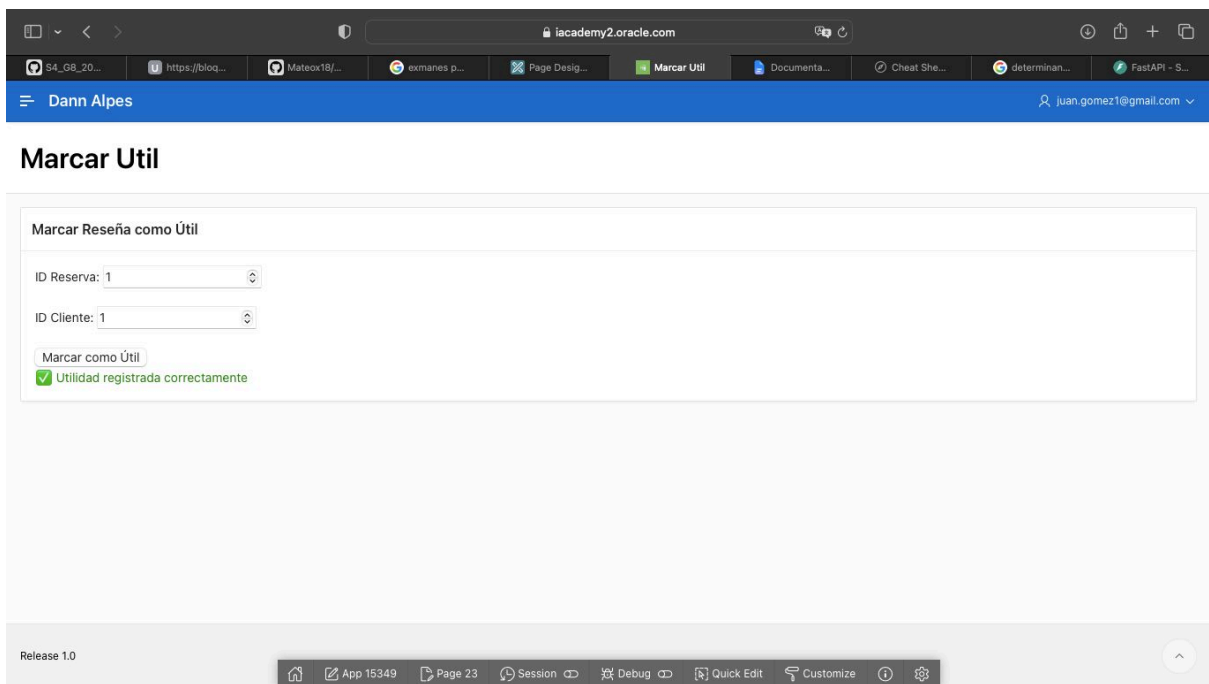
✓ Resena eliminada correctamente

Release 1.0

App 15349 Page 27 Session Debug Quick Edit Customize



Ranking de utilidad de las reseñas:



10. Conclusiones

MongoDB permitió modelar de manera flexible la información relacionada con reseñas, respuestas administrativas y votos útiles, el uso de estructuras embebidas facilitó consultas frecuentes y disminuyó la complejidad de acceso a la información.

Por otro lado, Oracle continuó administrando la información principal del sistema, permitiendo mantener una arquitectura adecuada para el problema planteado, dando paso a que finalmente el uso de FastAPI permitiera integrar correctamente Oracle, MongoDB y APEX dentro de una única solución funcional.